



FlowBoard

Technical & Product Documentation

A real-time collaborative task board — Flutter + Firebase, built end to end as a portfolio-grade engineering project.

Live application: <https://flowboard-app-7539.web.app>

Version documented: 1.0.0 (15 commits, initial scaffold through the final enhancement round)

Stack: Flutter (Web + Android), Riverpod, Firebase (Firestore, Auth, Hosting), Google Gemini

Automated tests: 56 passing (offline, plus a live Playwright E2E suite)

Table of Contents

1	Overview & Purpose
2	Feature Summary
3	Technology Stack
4	System Architecture
5	Data Model (Firestore Schema)
6	Authentication & Identity
7	Security Model & Permissions
8	Real-Time Collaboration Mechanics
9	Feature Deep Dives
10	Accessibility & PWA
11	UI/UX & Design System
12	Project Structure
13	Getting Started (Local Development)
14	Testing Strategy
15	Deployment & CI
16	Known Limitations & Future Work
17	Development History
18	Appendix A — Full Firestore Security Rules
19	Appendix B — Configuration & Environment Variables

1. Overview & Purpose

FlowBoard is a Trello-style collaborative task board built to demonstrate real, non-trivial engineering — not a CRUD demo. Every piece of "real-time" behavior in the app is backed by an actual Firestore listener chain, not a simulated or mocked interaction: dragging a card, another tab seeing it move instantly; two people editing the same card and getting a genuine conflict banner; a teammate's presence pill appearing and fading based on a live heartbeat.

The project was built iteratively across many rounds, starting from a static design reference and growing into a fully wired Firebase backend with real authentication, a genuine per-board membership and role system enforced by Firestore Security Rules, AI-assisted task breakdown, and a broad set of productivity features layered on top: labels, due-date tracking, bulk actions, CSV import/export, a cross-board "My Tasks" view, lightweight markdown, board templates, saved filters, keyboard shortcuts, and a real accessibility pass — each one verified against the live deployed app, not just written and assumed correct.

Design constraint carried through the entire project: no paid infrastructure. Every feature — including security-rules enforcement, avatar/attachment photo storage, CSV import/export, and invite links — was deliberately built to avoid requiring the Firebase Blaze (pay-as-you-go) plan, Cloud Functions, or Firebase Storage (which now requires Blaze even for free-tier usage). Where a paid approach would have been the "obvious" solution, a free client-side or rules-based alternative was used instead, and the trade-off is documented inline in this doc and in code comments.

The application is deployed and publicly reachable at <https://flowboard-app-7539.web.app>. It also builds for Android (`flutter build apk`); an iOS build was not attempted, since that requires macOS/Xcode and Firebase Console access unavailable in this project's development environment (see §16).

2. Feature Summary

Boards & Tasks

- Create, rename, recolor, archive, restore, and permanently (undoably) delete boards
- Start a new board from a template — Sprint board, Content calendar, Bug tracker — or blank
- Three-column workflow (To Do / In Progress / Done) with drag-and-drop reordering via a **fractional index**, so moving one card never rewrites its siblings
- Tasks carry title, description (markdown), priority, assignee, due date, labels, subtasks, small image attachments, comments, and an activity timeline
- Undoable task deletion (a 5-second snackbar grace window) and multi-select bulk move/delete

Finding Things

- **My Tasks** — everything assigned to you, aggregated across every board you belong to, grouped into Overdue / Due soon / Upcoming / No due date
- Per-board search with priority, assignee, overdue, and due-soon filters
- Named, saved filter combinations, re-appliable with one tap
- CSV export and import — a board's tasks round-trip losslessly through a spreadsheet
- Keyboard shortcuts: `n` new task, `/` search, `Esc` close

Real-Time Collaboration

- Live presence: see who else currently has a board open, with automatic staleness cleanup
- Conflict detection: a banner names who changed a card and when, computed from a real `updatedAt` / `updatedBy` comparison — not simulated
- Every mutation is a real Firestore write seen by every client through the same listener — no local-only optimistic state that could drift from the server

AI

- "AI subtasks": calls the Google Gemini API to break a vague task into a concrete checklist
- Falls back to a static suggestion list with no API key configured, so the feature always demos

Identity & Auth

- Sign in with Google, Apple, email/password, or as a guest (anonymous)
- Guest → real account upgrade via Google account linking
- Handles Firebase's ambiguous `invalid-credential` error with a dialog offering every real next step, and resolves the "email already has a password account" OAuth conflict by offering to link the credential automatically
- Reserved, globally-unique `@usernames`, race-safe against simultaneous claims and format-validated server-side (not just client-side)
- Real avatar photo upload (compressed client-side, stored inline — no Firebase Storage)

Invites & Sharing

- Invite by email, by `@username`, or a shareable `/join/<boardId>` link
- A paste-a-link dialog for joining directly inside the app

Permissions

- Real per-board membership — a board is invisible to anyone not on its member list, enforced by Firestore rules, not just hidden UI
- Owner / Editor / Viewer roles; a Viewer's every write path is blocked server-side
- The four seeded demo boards are visible to guests only, so a first-time visitor sees a populated, editable sandbox instantly, while a real account starts genuinely empty

Polish

- Notifications (assignment, comments, mentions) with "load more" pagination
- Per-board activity log with a 7-day task-completion velocity chart
- Light/Dark/System theme, onboarding flow, branded splash screen
- Installable PWA — filled-in manifest, real icons, registered service worker
- Accessibility pass — tooltip/semantic labels on icon-only buttons, WCAG AA contrast checked (and fixed where it failed) rather than assumed

3. Technology Stack

LAYER	CHOICE	WHY
UI framework	Flutter (Dart)	Single codebase targeting Web (primary deployment) and Android; compiles to a real native-feeling app, not a wrapped webview.
State management	Riverpod (<code>flutter_riverpod</code>)	<code>StreamProvider</code> / <code>StateNotifierProvider.autoDispose</code> map cleanly onto Firestore's snapshot-listener model, and <code>.family</code> providers scope state per board/task/user id.
Backend	Firebase Firestore	Real-time listeners are the actual sync mechanism — no custom WebSocket server, no polling.
Auth	Firebase Authentication	Google, Apple, email/password, and anonymous providers, all first-class.
Hosting	Firebase Hosting	Static web build deployment with a single-page-app rewrite rule.
AI	Google Gemini API	Free-tier accessible via a plain REST call from the client, compile-time key injection.
Image handling	<code>image_picker</code> + <code>image</code> (pure Dart)	Client-side resize/compress/encode avoids needing Firebase Storage for avatars and task attachments alike.
Links	<code>url_launcher</code>	Opens tappable markdown links from task descriptions/comments in a new tab.
Local persistence	<code>shared_preferences</code>	Onboarding-seen flag and per-board saved search filters — genuinely personal, device-local settings that don't need to sync.

Key packages (pubspec.yaml)

```
flutter_riverpod: ^2.6.1      # state management
google_fonts: ^6.2.1
firebase_core: ^3.8.0
cloud_firestore: ^5.5.0
firebase_auth: ^5.3.3
http: ^1.2.2                 # Gemini REST calls
shared_preferences: ^2.3.3    # onboarding flag, saved filters
intl: ^0.19.0
uuid: ^4.5.1
image_picker: ^1.1.2         # avatar + attachment photo picking
image: ^4.2.0                # client-side compression
url_launcher: ^6.3.1         # tappable markdown links

dev:
fake_cloud_firestore: ^3.1.0  # provider unit tests without real Firebase
```

```
firebase_auth_mocks: ^0.14.1  
flutter_lints: ^6.0.0
```

4. System Architecture

FlowBoard has no custom backend server. All persistence, real-time sync, and access control run through Firestore and its Security Rules; all compute (aside from the client itself) is the Gemini API called directly from the browser. This keeps the entire system free to run and easy to reason about, at the cost of pushing more logic — especially authorization and, in this iteration, cross-board aggregation — into the client and declarative security rules rather than server code.

Layering

- **Screens** (`lib/screens/`) — one file per full-page route.
- **Widgets** (`lib/widgets/`) — reusable pieces shared across screens (member avatar, task card, invite sheet, rich-text renderer, shared back button, etc.).
- **Providers** (`lib/providers/`) — all Firestore/Auth interaction. Screens never call `cloud_firestore` directly except through a provider.
- **Models** (`lib/models/`) — plain immutable Dart classes with no Firestore knowledge of their own.
- **Data** (`lib/data/`) — Firestore document mappers, CSV import/export, board templates, saved-filter persistence, markdown/mention token parsing, and the one-time demo-data seed. Parsing logic here is deliberately pure (no I/O, no widgets) so it's independently unit-testable.
- **Services** (`lib/services/`) — the Gemini HTTP client, deep-link capture, and web-only conditional-import helpers (URL strategy, file export/import).

Real-time sync, concretely

A typical write path looks like: user drags a card → `BoardTasksNotifier.moveTask()` computes a new fractional order → one Firestore `update()` call → every open client (including the one that made the move) receives the change back through the same `snapshots()` listener that populates the board. There is no separate "optimistic local update" — what you see after dropping the card is the same round trip every other viewer sees, just fast enough to feel instant.

Combining data across boards, safely

The "My Tasks" cross-board view (§9) is the one place FlowBoard needed data from *every* board a user belongs to at once. The naive approach — a Firestore `collectionGroup('tasks')` query — was deliberately avoided: it would hit the exact same class of bug that once broke the boards list entirely (§7), since the `tasks` security rule depends on `get()`-ing the parent board, which a list query can't statically prove holds for every result. Instead, `myTasksProvider` watches the existing, already-safe `boardTasksProvider` for every board in the user's board list and combines the results client-side — the same per-board query every board screen already runs, just several at once. The cost is one live listener per board while the My Tasks screen is open, a reasonable trade for a typical handful of boards.

5. Data Model (Firestore Schema)

Top-level collections

COLLECTION	DOCUMENT ID	PURPOSE
<code>boards</code>	auto or fixed (seed ids <code>b1</code> – <code>b4</code>)	One document per board.
<code>boards/{id}/tasks</code>	auto	Subcollection: one document per task card.
<code>boards/{id}/presence</code>	uid	Subcollection: live-viewer heartbeats.
<code>users</code>	uid	One document per account (profile, email, username, photo, bio).
<code>users/{uid}/notifications</code>	auto	Subcollection: per-user notification feed.
<code>usernames</code>	lowercase username	Reservation map: <code>{ uid }</code> , keyed by the name itself so uniqueness is a document-existence check.
<code>meta</code>	<code>seed</code>	One-time marker doc guarding the demo-data seed against double-seeding from concurrent app launches.

`boards/{boardId}` fields

FIELD	TYPE	NOTES
<code>name</code>	string	
<code>color</code>	int (ARGB32)	
<code>ownerId</code>	string (uid)	Always resolves to role <code>owner</code> regardless of the <code>roles</code> map.
<code>members</code>	array<map>	Denormalized display list: <code>{id, name, initials, color}</code> per member.
<code>memberIds</code>	array<string>	Kept in sync with <code>members</code> ; exists purely so security rules can do a cheap <code>uid in memberIds</code> check.
<code>roles</code>	map<uid, string>	<code>"editor"</code> or <code>"viewer"</code> ; missing entry defaults to editor.
<code>isDemo</code>	bool	True only on the four seeded boards; makes them guest-visible.
<code>linkJoinEnabled</code>	bool	Toggle controlling whether the shareable invite link currently admits new members.

<code>archived</code>	bool	Hides the board from the main list without deleting it.
<code>updatedAt</code>	timestamp	Server timestamp, bumped on every board-level write.

`boards/{id}/tasks/{taskId}` **fields**

FIELD	TYPE	NOTES
<code>title</code> , <code>description</code>	string	Description supports the lightweight markdown grammar (§9).
<code>priority</code>	string	<code>low</code> / <code>medium</code> / <code>high</code>
<code>column</code>	string	<code>todo</code> / <code>inProgress</code> / <code>done</code>
<code>order</code>	double	Fractional index within the column — see §8.
<code>assignee</code>	map	Denormalized member snapshot.
<code>dueDate</code>	timestamp?	Optional; drives overdue/due-soon status.
<code>labels</code>	array<string>	Preset label names (e.g. <code>Bug</code> , <code>Feature</code>); each has a fixed color defined client-side.
<code>attachments</code>	array<string>	Base64-encoded compressed JPEGs, capped at ~120 KB each, 3 per task.
<code>subtasks</code>	array<map>	<code>{id, text, done}</code> ; AI-generated or manual.
<code>comments</code>	array<map>	<code>{id, author, time, body}</code> ; body supports markdown + @mentions.
<code>activity</code>	array<map>	Append-only timeline entries (created, moved, commented, imported); the velocity chart derives from "moved to Done" entries.
<code>updatedAt</code> / <code>updatedBy</code>	timestamp / map	Drives conflict-banner detection; every write must attribute itself truthfully (enforced by rules).

`users/{uid}` **fields**

FIELD	TYPE	NOTES
<code>id</code> , <code>name</code> , <code>initials</code> , <code>color</code>	—	Core <code>Member</code> shape, duplicated wherever a Member snapshot is embedded elsewhere.
<code>email</code>	string?	Lowercase-normalized; present only for non-anonymous accounts.
<code>username</code>	string?	Mirrors the key that owns this uid in the <code>usernames</code> collection.

photoBase64

string? A compressed JPEG, base64-encoded, capped at ~180 KB.

bio

string? Free-text, capped at 160 characters client-side.

6. Authentication & Identity

Sign-in methods

- **Google** — `signInWithPopup(GoogleAuthProvider())`
- **Apple** — `signInWithPopup(OAuthProvider('apple.com'))` with `email` and `name` scopes requested
- **Email / password** — standard Firebase email-password auth, plus account creation and password reset
- **Guest** — `signInAnonymously()` ; guests are assigned a random persona (name/initials/color) so a handful of guest tabs look like distinct collaborators

The ambiguous-credential problem

Modern Firebase Auth deliberately returns the same `invalid-credential` error code for "wrong password" and "no account exists" (an email-enumeration protection). Instead of guessing, the sign-in screen presents a dialog offering every real path forward: create a new account, try Google, or try Apple. If an OAuth attempt then reveals the email is already registered with a password, the email field is pre-filled and the app offers to link the OAuth credential to the existing account once the correct password is entered.

Profile resolution

`currentMemberProvider` is the single place a signed-in Firebase `User` becomes a FlowBoard `Member`. On first sign-in it creates the `users/{uid}` document; on every sign-in it opportunistically backfills the `email` field for accounts created before that field existed. The resolved value is mirrored into `currentMemberStateProvider` so the rest of the app can read the current member synchronously.

7. Security Model & Permissions

Every access-control decision in FlowBoard is enforced in Firestore Security Rules, not just hidden in the UI. This section explains the model; the full rules listing is in Appendix A.

Membership

A board carries `memberIds` and a `roles` map. Reading, writing, or even knowing a board exists requires `request.auth.uid in memberIds` — with two narrow, deliberate exceptions below. A `viewer` member can read the board and its tasks but every write path is rejected server-side. The board owner can always change the `roles` map; a plain editor cannot promote or demote anyone.

Exception 1 — demo boards, guests only

The four seeded boards carry `isDemo: true`. The rules grant read/write access to them for *anonymous* sign-ins only — a real account never sees them, preserving both the "click the link, see a populated board instantly" demo experience and a genuinely private-by-default model for real accounts.

Exception 2 — invite links

A board with `linkJoinEnabled: true` can be read by any signed-in user, and a non-member is allowed to *update* it — but only to add their own uid to `memberIds`, assign only their own key in `roles` to `"editor"`, and touch no other field. Verified directly against the rules: a join attempt that tries to self-promote to owner, add an unrelated uid, or sneak in a board-name change in the same write is rejected with `PERMISSION_DENIED`.

Username reservations

Uniqueness is enforced by Firestore's own write semantics: the `usernames/{name}` document's `create` operation only succeeds if it doesn't already exist, so two users racing to claim the same name can't both win. The rule additionally validates the *format* of the name itself server-side (`name.matches('^[a-z0-9_]{3,20}$')`) and restricts the document to exactly the `uid` field (`hasOnly(['uid'])`) — a hardening pass added specifically to stop someone from scripting past the app's own client-side validation and reserving junk or oversized document ids directly against the API.

Identity integrity

Every task write must carry `updatedBy.id == request.auth.uid` — you cannot spoof another user as the author of a change, which is what makes conflict-detection banners and the activity log (and the velocity chart derived from it) trustworthy.

8. Real-Time Collaboration Mechanics

Fractional-index drag-and-drop

Moving a card to a new position computes a new `order` value as the midpoint between its new neighbors' existing order values (or ± 1000 at either end of a column). Reordering one card is always a single-document write — no sibling documents are touched, and the approach scales to arbitrarily long columns. Bulk moves (§9) use the same scheme, appending each selected task after the target column's current last card in one batched write.

Presence

Firestore has no native "this client disconnected" signal. Presence is therefore heartbeat-based: while a board screen is open, a `PresenceHeartbeat` writes a refreshed timestamp to `boards/{id}/presence/{uid}` every 12 seconds and deletes it on clean dispose. A companion `PresenceNotifier` re-filters the visible list every 8 seconds against a 25-second staleness cutoff, so a tab that closed uncleanly still drops off within about half a minute.

Conflict detection

When a task detail sheet opens, it captures the task's current `updatedAt` as a baseline. If the live Firestore listener later reports a newer `updatedAt` attributed to a different `updatedBy`, a banner appears naming who made the change and when — a real comparison against live server data.

9. Feature Deep Dives

Task labels & due-date tracking

Seven preset labels (Bug, Feature, Design, Backend, Frontend, Docs, Urgent), each with a fixed color defined once in `label_colors.dart` — a closed set rather than free-form tagging, so label chips stay visually consistent across a board. `TaskCard.isOverdue` / `isDueSoon` are computed getters (a task with a past due date still in an open column is overdue; within 2 days is due soon; anything already in Done is never flagged) reused identically by the task card badge, the detail-sheet due-date line, the search screen's quick filters, and the My Tasks screen's grouping — one definition of "overdue," not four.

Bulk actions

A selection-mode toggle on the board screen replaces each card's drag/tap behavior with a checkbox overlay; a bottom action bar offers "Move to" (any column) or "Delete" for every selected task, both as single batched Firestore writes.

Undo-on-delete

Deleting a task or a board doesn't delete immediately — it shows a snackbar with an Undo action via the app-level `ScaffoldMessengerKey` (so the undo window survives navigating away from the sheet that triggered it) and only performs the actual Firestore delete once that snackbar closes without Undo being tapped. Board deletion additionally snapshots the board document and every task document's raw data beforehand, so Undo restores them exactly rather than reconstructing an approximation.

CSV export & import

`buildBoardCsv()` and `parseBoardCsv()` are pure functions (in `lib/data/csv_export.dart`) that round-trip a board's tasks through a standard quoted CSV — export writes Title/Description/Column/Priority/Labels/Assignee/Due date/subtask and comment counts; import reads by header name (not position), requires only a `Title` column, and matches "Assignee" against the board's current members by exact display name, falling back to whoever ran the import on no match. The actual browser file download/picker (`lib/services/file_export.dart`, `file_import.dart`) uses the same conditional-import pattern as the web-only URL strategy, so `flutter test` on the VM target never tries to compile a browser-only API.

@mentions & lightweight markdown

Comment text is parsed by a single pure function, `parseRichText()` (`lib/data/rich_text_tokens.dart`), into typed tokens — plain, `@mention`, `**bold**`, `*italic*`, ``code``, and `[text](url)` — which a small widget then maps to styled/tappable `TextSpan`s. Mentions are visually highlighted immediately but resolved server-side: `BoardTasksNotifier.addComment()` extracts `@handle` tokens, looks each up against the reserved-username collection (the same lookup invites use), and restricts notification to people who are actually on that board — mentioning someone who can't see the task notifies no one.

Task attachments

Small image attachments reuse the exact avatar-photo pipeline (`image_picker` → client-side resize/compress via `package:image` → base64, no Firebase Storage), capped at ~120 KB each with a 3-per-task limit to keep a heavily-used task document comfortably under Firestore's 1 MiB document size limit alongside its comments, subtasks, and activity log.

Board templates

Three starter presets (Sprint board, Content calendar, Bug tracker) are plain client-side data (`lib/data/board_templates.dart`); picking one at board-creation time seeds real task documents — assigned to the creator, in the right columns, with a real fractional order — in the same batched write as the board document itself.

My Tasks (cross-board view)

See §4's "Combining data across boards, safely" — `myTasksProvider` watches every board the user belongs to and every one of those boards' task streams, filters to tasks assigned to the current user, and sorts overdue-first/due-soon-next/then-by-due-date, all client-side over already-live data rather than a new Firestore query shape.

Saved filters

A named combination of priority/assignee/overdue/due-soon filters, persisted via `shared_preferences` keyed per board (`lib/data/saved_filters.dart`) — a deliberately device-local, personal convenience rather than Firestore-synced board data, since assignee ids only make sense within one board's own member list anyway.

Keyboard shortcuts

`n` opens a new task in the active column; `/` opens search. Implemented with Flutter's `CallbackShortcuts` (which integrates with the same focus-tree key dispatch a focused text field uses to consume its own typing) rather than a raw `HardwareKeyboard` hook. An earlier version used the raw hook with a manual "is a text field focused" guard; live verification caught that the guard didn't reliably detect focus on Flutter web, so typing a literal `n` or a URL containing `/` into a comment could hijack the keystroke into opening a new task or navigating to search. The fix also had to account for `showModalBottomSheet` keeping the underlying screen's shortcuts reachable even while its own sheet has focus (unlike a full route push) — the board screen now explicitly tracks "a sheet is open" and gates both shortcuts on it, which is arguably the more correct behavior regardless: pressing `n` shouldn't open a second new-task sheet over the one already open.

10. Accessibility & PWA

Accessibility audit — what was actually checked

Rather than assume icon-only buttons and color choices were fine, this pass measured them:

- **Semantic labels:** every icon-only interactive element (back buttons, search, bulk-select toggle, delete, avatar upload, notifications, board menu) was wrapped in `Tooltip` — which both shows a hover label on desktop/web and gives assistive tech a real label an icon alone doesn't provide. Six near-identical private `_BackButton` classes duplicated across separate screen files were consolidated into one shared `AppBackButton` widget specifically so this fix applied everywhere at once rather than needing six separate edits.
- **Color contrast:** computed WCAG relative-luminance contrast ratios for every priority-tag color pairing actually used in the app. The pairing a seeded demo task's title literally calls out ("Audit color contrast: Priority tags fail 4.5:1 on the dark surface") turned out to measure ~7:1 — fine. The real failure was elsewhere and unrelated to that seed data: white text on `colors.accent` in two "selected priority pill" components measured as low as ~1.8:1 against the required 4.5:1, in *both* light and dark mode, across all three priorities. Fixed by swapping to a dark, near-black label color, which clears 6:1+ against every priority's accent color in both themes.

PWA installability

Audited rather than assumed complete: `web/manifest.json` already had a real name, icons (192/512, including maskable variants), theme/background colors, and `display: standalone`; the Flutter web build already emits and auto-registers `flutter_service_worker.js`. No changes were needed — the app was already a fully installable PWA once served over HTTPS (which Firebase Hosting provides by default).

11. UI/UX & Design System

Visual design was sourced from a Claude-generated design export cross-referenced against PNG screenshots for individual screens. Tokens were ported into `lib/theme/`:

FILE	CONTENTS
<code>app_colors.dart</code>	Full light/dark palette, priority colors, brand primary (<code>#5B45E0</code>).
<code>app_text_styles.dart</code>	Named text styles matching the design system's type scale.
<code>app_metrics.dart</code>	Shared corner radii and spacing constants.
<code>app_theme.dart</code>	Assembles the above into Flutter <code>ThemeData</code> for light and dark mode.
<code>priority_colors.dart</code>	Per-priority accent/background/text color triples — the pairings audited for contrast in §10.
<code>label_colors.dart</code>	The seven fixed preset-label colors.

The brand mark (`FlowBoardLogo`) is drawn as vector shapes — a rounded-square background behind three vertical bars — rather than an image asset, so it scales cleanly from a 20px header wordmark up to a 96px profile corner badge and stays trivially theme-aware. The same "plain containers, no image assets" style extends to the velocity chart (§9) and the boards-list empty-state illustration — hand-rolled rather than pulling in a charting/illustration package.

12. Project Structure

```
lib/
├─ main.dart                # entry point; captures deep link, boots Firebase
├─ firebase_options.dart    # generated by flutterfire_cli
├─ data/
│   ├─ firestore_mappers.dart # Firestore doc ⇄ model conversions
│   ├─ firestore_seed.dart   # one-time demo-board seed
│   ├─ mock_data.dart, personas.dart
│   ├─ csv_export.dart       # buildBoardCsv / parseBoardCsv (pure)
│   ├─ board_templates.dart  # Sprint/Content calendar/Bug tracker presets
│   ├─ saved_filters.dart    # per-board named filter persistence
│   └─ rich_text_tokens.dart # markdown + @mention parsing (pure)
├─ models/                  # plain immutable data classes
│   └─ board.dart, task_card.dart, member.dart, comment.dart, ...
├─ providers/               # all Firestore/Auth logic
│   ├─ auth_provider.dart    # sign-in methods, error mapping
│   ├─ profile_provider.dart  # profile resolution, username, avatar photo
│   ├─ boards_provider.dart   # board CRUD, membership, presence, templates
│   ├─ board_tasks_provider.dart # task CRUD, drag reorder, bulk ops, AI, CSV import
│   ├─ my_tasks_provider.dart # cross-board "assigned to me" aggregation
│   ├─ notifications_provider.dart
│   └─ theme_provider.dart
├─ screens/                 # one file per full-page route
│   ├─ splash_screen.dart, root_gate.dart, auth_gate.dart
│   ├─ sign_in_screen.dart, onboarding_screen.dart
│   ├─ boards_list_screen.dart, board_detail_screen.dart
│   ├─ board_settings_screen.dart, archived_boards_screen.dart
│   ├─ task_detail_sheet.dart, search_screen.dart
│   ├─ activity_screen.dart, notifications_screen.dart
│   └─ my_tasks_screen.dart, profile_screen.dart
├─ widgets/                 # reusable shared UI
│   ├─ board_card.dart, board_column_view.dart, task_card_widget.dart
│   ├─ member_avatar.dart, member_role_row.dart, presence_bar.dart
│   ├─ invite_sheet.dart, new_task_sheet.dart, empty_states.dart
│   ├─ flowboard_logo.dart, oauth_icons.dart, priority_tag.dart
│   └─ app_back_button.dart, rich_text_content.dart, task_drag_data.dart
├─ services/
│   ├─ gemini_service.dart    # Gemini REST client
│   ├─ deep_link.dart         # /join/{id} URL capture
│   ├─ url_strategy*.dart     # web-only, conditionally imported
│   ├─ file_export*.dart      # CSV download (web-only)
│   └─ file_import*.dart      # CSV file picker (web-only)
└─ theme/                   # design tokens (see §11)

test/
├─ board_tasks_provider_test.dart, boards_provider_test.dart
├─ my_tasks_provider_test.dart, profile_provider_test.dart
├─ csv_export_test.dart, rich_text_tokens_test.dart, saved_filters_test.dart
├─ task_card_test.dart, member_avatar_test.dart, widget_test.dart
└─ test/e2e/                # Playwright checks against the live deployed app

firestore.rules              # full security rules (Appendix A)
firestore.indexes.json
```

```
web/index.html          # includes the pre-Flutter HTML splash
.github/workflows/ci.yml # analyze + test + build on every push/PR
```

13. Getting Started (Local Development)

Prerequisites

- Flutter SDK (Dart ^3.11.1 or newer)
- A Firebase project with Firestore and Authentication enabled
- The Firebase CLI (`npm i -g firebase-tools`) and `flutterfire_cli`
- (Optional) a Google Gemini API key for real AI subtasks — see aistudio.google.com/apikey

Setup

```
# Install dependencies
flutter pub get

# Generate lib/firebase_options.dart for your own Firebase project
flutterfire configure

# Enable sign-in providers you want in Firebase Console → Authentication:
#   Google, Apple (needs an Apple Developer "Services ID" configured
#   with a return URL of https://<project>.firebaseapp.com/__/auth/handler),
#   Email/Password, Anonymous

# Deploy security rules
firebase deploy --only firestore:rules

# Run locally (web)
flutter run -d chrome --dart-define=GEMINI_API_KEY=your_key_here
```

Running tests

```
flutter analyze
flutter test

# Live-browser checks against a deployed URL (see test/e2e/README.md)
cd test/e2e && npm install && npx playwright install chromium && npm test
```

14. Testing Strategy

Two complementary layers of testing were used throughout development:

Automated (unit & widget tests) — 56 passing

Provider logic is tested against `fake_cloud_firestore` and `firebase_auth_mocks` — a real (in-memory) Firestore API surface without needing a live project. This covers fractional-index math, conflict-attribution on moves, bulk move/delete, CSV round-tripping, board-template seeding, notification triggers (including @mention resolution restricted to actual board members), the cross-board My Tasks aggregation and its sort order, saved-filter persistence, markdown/mention token parsing, due-date status logic, and username-reservation race conditions. Widget tests cover key screens using `ProviderScope` overrides.

Manual/live (Playwright)

Because so much of the app's value is in genuinely-real multi-user behavior, many features were verified against the live deployed app using Playwright browser automation. This caught several real defects unit tests alone would have missed — not hypothetically, but in this project's actual history:

- Firestore rejecting an entire unfiltered collection query once its rule depended on per-document fields — the boards list going silently empty for every user right after a rules deploy
- A stale build cache silently dropping a web plugin's registration
- An avatar photo with transparent pixels rendering as a see-through hole instead of falling back to the member's color
- A sign-in screen layout bug (`CrossAxisAlignment.stretch` forcing a fixed-size logo icon into a full-width banner) that only showed up visually, not in any test assertion
- The keyboard-shortcut regression described in §9 — caught specifically by typing a real URL into a live comment field during verification, not by testing the shortcuts in isolation

15. Deployment & CI

```
# Build the web bundle (release, with the AI key baked in)
flutter build web --release --dart-define=GEMINI_API_KEY=your_key_here

# Deploy hosting only
firebase deploy --only hosting

# Deploy security rules only (after any firestore.rules change)
firebase deploy --only firestore:rules
```

`.github/workflows/ci.yml` runs `flutter analyze`, `flutter test`, and a release web build on every push and pull request — a green check on the repo without needing to run anything locally.

Migration note: when the membership/roles model was introduced, existing production board documents needed a one-time backfill (`memberIds`, `roles`, `isDemo` for the seed boards) run *before* the stricter rules were deployed — otherwise every board would have failed the new rule's `uid in memberIds` check immediately. Any future rules change that gates on a new field needs the same care: backfill under the old rules, verify, then deploy the new rules.

16. Known Limitations & Future Work

LIMITATION	REASON
No iOS build verified	Requires macOS + Xcode and Firebase Console access to register an iOS app (for <code>GoogleService-Info.plist</code>), neither available in this project's development environment. The Android build is verified (<code>flutter build apk</code> succeeds).
Gemini API key has no website-restriction	Current Gemini key types support API-scope restriction (applied) but not HTTP-referrer restriction. A backend proxy would fully solve this but requires the paid Blaze plan.
Avatar photos and task attachments are size-capped	~180 KB / ~120 KB×3 respectively, stored inline on their document to avoid the Blaze-plan-only Firebase Storage.
CSV import assignee matching is name-based	A spreadsheet cell can't carry a real member id; exact-name matching against current board members, with a same-importer fallback, is the practical alternative.
Saved filters are device-local	Stored via <code>shared_preferences</code> , not Firestore, so they don't follow a user to a different device — a deliberate trade against adding another synced collection for a personal-convenience feature.
No push notifications	In-app notification feed only; browser/mobile push would need FCM wiring, not yet built.

17. Development History

High-level summary of the project's commit history, oldest to newest:

1. **Initial commit** — full Flutter + Firebase scaffold: real-time sync, presence, conflict detection, drag-and-drop, Google/email/guest auth, notifications, Gemini AI subtasks.
2. **Membership & roles** — per-board `memberIds` / `roles`, task deletion, archived-board recovery, anonymous-to-Google account linking.
3. **Hotfix** — boards list breaking under the new membership-gated rules (unfiltered collection query rejected outright); split into two provably-safe queries.
4. **Sign-in UX fixes** — resolved the ambiguous-credential dead end; added a profile avatar color picker.
5. **Guest-only demo boards** — restricted the four seed boards to anonymous sessions only, both client-side and via rules.
6. **Real invite links** — `linkJoinEnabled` toggle, narrowly-scoped self-join security rule, `/join/{id}` deep link handling.
7. **Avatar photos & usernames** — live per-uid photo watching, race-safe username reservation, invite-by-username.
8. **Splash screen, sign-in polish** — branded HTML + in-app splash, sign-in screen polish to match a design reference.
9. **OAuth error-message accuracy fix** — surfaced during live testing.
10. **"Join with an invite link" wired up** — replaced a leftover placeholder with a real paste-a-link dialog.
11. **Apple sign-in** — replaced an earlier Microsoft/Azure attempt entirely with Apple, per final product direction.
12. **Enhancement round 1** — task labels, due-date tracking, bulk move/delete, @mention notifications, CSV export, undo-on-delete, GitHub Actions CI, a formalized Playwright E2E suite.
13. **Enhancement round 2** — task attachments, CSV import, keyboard shortcuts, a 7-day velocity chart, and a real accessibility pass (contrast fixes, semantic labels) — including verifying the PWA manifest/service worker were already complete rather than assuming they needed work.
14. **Enhancement round 3 (final)** — board templates, the My Tasks cross-board view, lightweight markdown, saved filters — and a fix for a keyboard-shortcut regression the previous round introduced, caught by live verification rather than assumed correct.

18. Appendix A — Full Firestore Security Rules

```
rules_version = '2';

service cloud.firestore {
  match /databases/{database}/documents {

    function isSignedIn() {
      return request.auth != null;
    }

    function isSelf(uid) {
      return request.auth.uid == uid;
    }

    function isAnonymousUser() {
      return isSignedIn() && request.auth.token.firebase.sign_in_provider == 'anonymous';
    }

    match /meta/{docId} {
      allow read, write: if isSignedIn();
    }

    match /users/{uid} {
      allow read: if isSignedIn();
      allow write: if isSignedIn() && isSelf(uid);

      match /notifications/{notifId} {
        allow read, update, delete: if isSignedIn() && isSelf(uid);
        allow create: if isSignedIn();
      }
    }

    match /usernames/{name} {
      allow read: if isSignedIn();
      allow create: if isSignedIn()
        && request.resource.data.uid == request.auth.uid
        && request.resource.data.keys().hasOnly(['uid'])
        && name.matches('^[-a-z0-9_]{3,20}$');
      allow delete: if isSignedIn() && resource.data.uid == request.auth.uid;
      allow update: if false;
    }

    match /boards/{boardId} {
      function isDemoBoard() {
        return resource.data.isDemo == true && isAnonymousUser();
      }
      function isMember() {
        return isSignedIn() && request.auth.uid in resource.data.memberIds;
      }
      function isMemberOfNew() {
        return isSignedIn() && request.auth.uid in request.resource.data.memberIds;
      }
      function myRole() {
```

```

    return resource.data.roles[request.auth.uid];
}
function rolesUnchanged() {
    return request.resource.data.roles == resource.data.roles;
}
function isOwner() {
    return isSignedIn() && resource.data.ownerId == request.auth.uid;
}
function isLinkJoinEnabled() {
    return resource.data.linkJoinEnabled == true;
}
function isSelfJoiningViaLink() {
    return isLinkJoinEnabled()
        && !(request.auth.uid in resource.data.memberIds)
        && request.resource.data.memberIds == resource.data.memberIds.concat([request.auth.uid])
        &&
request.resource.data.roles.diff(resource.data.roles).affectedKeys().hasOnly([request.auth.uid])
        && request.resource.data.roles[request.auth.uid] == 'editor'
        && request.resource.data.diff(resource.data).affectedKeys().hasOnly(['members',
'memberIds', 'roles', 'updatedAt']);
}

allow read: if isSignedIn() && (isDemoBoard() || isMember() || isLinkJoinEnabled());
allow create: if isSignedIn()
    && isMemberOfNew()
    && request.resource.data.ownerId == request.auth.uid;
allow update: if isSignedIn()
    && (isDemoBoard()
        || (isMember() && myRole() != 'viewer' && (rolesUnchanged() || isOwner())))
        || isSelfJoiningViaLink());
allow delete: if isOwner();

match /presence/{uid} {
    allow read: if isSignedIn();
    allow write: if isSignedIn() && isSelf(uid);
}

match /tasks/{taskId} {
    function board() {
        return get(/databases/${database}/documents/boards/${boardId}).data;
    }
    function isDemoTaskBoard() {
        return board().isDemo == true && isAnonymousUser();
    }
    function isBoardMember() {
        return isSignedIn() && request.auth.uid in board().memberIds;
    }
    function isBoardViewer() {
        return board().roles[request.auth.uid] == 'viewer';
    }

    allow read: if isSignedIn() && (isDemoTaskBoard() || isBoardMember());
    allow create, update: if isSignedIn()
        && (isDemoTaskBoard() || (isBoardMember() && !isBoardViewer()))
        && request.resource.data.updatedBy.id == request.auth.uid;
    allow delete: if isSignedIn() && (isDemoTaskBoard() || (isBoardMember() &&

```

```
!isBoardViewer()));  
    }  
  }  
}  
}
```

19. Appendix B — Configuration & Environment Variables

VARIABLE / FILE	PURPOSE
<code>--dart-define=GEMINI_API_KEY=...</code>	Compile-time constant read via <code>String.fromEnvironment</code> ; never committed to source. Without it, AI subtasks fall back to a static list.
<code>lib/firebase_options.dart</code>	Generated by <code>flutterfire configure</code> ; contains per-platform Firebase app config (API keys, project id, app ids). These are client identifiers, not secrets — Firestore/Auth security comes entirely from Security Rules, not from hiding this file.
<code>firestore.rules</code>	Deployed via <code>firebase deploy --only firestore:rules</code> . See Appendix A for the full listing.
Firebase Console → Authentication → Sign-in method	Google, Apple, Email/Password, and Anonymous must each be explicitly enabled (and saved) for their respective sign-in buttons to function.
Apple provider (Firebase Console)	Requires an Apple Developer "Services ID" with Sign in with Apple enabled and a return URL of <code>https://<project>.firebaseapp.com/__/auth/handler</code> , plus the generated key ID/team ID/private key entered into Firebase's Apple provider settings.
<code>FLOWBOARD_URL</code> (env var)	Overrides the Playwright E2E suite's target URL (default: the live production URL) — set it to point the suite at a local build or a hosting preview channel instead.

End of document. FlowBoard — a portfolio project built to demonstrate real-time collaborative systems engineering with Flutter and Firebase, without relying on any paid infrastructure.